

LIGO for TypeScript developers on Layer-1

Christian Rinderknecht

`Christian.Rinderknecht@trili.tech`

TriliTech

17 April 2025

Show and Tell

A short history of LIGO

- I did my PhD at INRIA under the supervision of Michel Mauny.
- I joined Nomadic in July 2018, where I started designing a smart contract language, similar to Pascal, that would be compiled to Michelson.
- My then-colleague Gabriel Alfour also wanted to make programming smart contracts on Tezos easier, but with a bottom-up approach: building a set of macros that expand into Michelson.
- We joined our efforts in 2019 into what became LIGO.
- I contributed the design and the front-end, he did the type-checker and code generator (50-50 in lines of code).

A short history of LIGO

- The idea was for LIGO to offer different concrete syntaxes inspired from mainstream programming languages.
- Those LIGO languages were meant to be domain-specific, but not overly so: supporting special literals (natural numbers, mutez, for examples), but taking most Tezos-specific types and functions from a library.
- The front-end had to lower their input to the common pipeline, while sharing as much code as possible in the processing of the different syntaxes.
- The first syntax was **PascaLIGO**, as I thought it would suit an imperative style, and lots of keywords would make the code more explicit, given the high stakes.
- Next, turmoil in the nascent ecosystem lead me to design **CameLIGO**. Then came **ReasonLIGO**, **JsLIGO** (which is today's topic), and even **PyLIGO**, which was never plugged in.

- Nowadays, only two LIGO syntaxes remain: **CameLIGO** and **JsLIGO**. The former is a strict subset of **OCaml**, if we leave aside the Tezos-specific literals, a local type definition construct, and attributes (they occur in prefix position in LIGO).
- **JsLIGO** started also as a subset but evolved and became syntactically incompatible with its model, **TypeScript**. For example, a proposal for pattern matching in **TypeScript** was implemented in **JsLIGO**.
- The front-end is made, in order, of a preprocessor, a lexer, two parsers and the first tree transformation before type-inference. Each pass can be built as a standalone executable for testing.
- I wrote the preprocessor because, at the time, we had no time to design and implement a module system, together with a build system.
- Unfortunately, it became very popular, even when LIGO got itself a module system, because it enabled conditional compilation (often for new type definitions of the storage, depending on the version of the standard being implemented).

Preprocessor and Lexer

- The preprocessor is not trivial, as it needs to understand UTF-8 encoded glyphs (in comments only) and the kind of comments and strings that depend on the LIGO variant.
- The lexer is also a functor parameterised by a lexer that is specific to a given LIGO syntax (comments, strings, keywords etc.). Error reporting has to be aware of linemarkers left by the preprocessor. Some style constraints are enforced (like a linter) for the sake of clarity.
- The lexer is specified as an input to `ocamllex`.
- Until `JsLIGO` was added to the party, the lexer signature was a function that returned a single token, which the parser called on demand. Syntactic constructs difficult to parse sometimes rely on lexer hacks, that is, having the parser informing the lexer of the syntactic left-context. Instead, I had the lexer run on all the (preprocessed) input, before applying a series of self-passes on the tokens — injecting virtual tokens when needed to help the parser. This is not as expressive as having the syntactic context, but rational approximations work quite well in practice.

- Perhaps the most interesting self-pass on the tokens is found with JsLIGO:

```
1 :      const f = ([x,y]) ...
```

could be a parenthesised pair or the start of an arrow function (a shift-reduce conflict). Analysing the lexical right-context in the grammar helps disambiguate the two cases, and the lexer injects a ES6FUN token in front of a function:

```
CONST IDENT EQ ES6FUN LPAR ...
```

The parser knows a function is coming next upon reading the virtual token ES6FUN (no more conflict).

- A classic one is needed to handle ">>" in

```
1 :      type t = u<v<w>>
```

for which a self-pass injects a Zero-Width SPace (ZWSP) virtual token between the two closing chevrons:

```
TYPE IDENT EQ IDENT LCHEV IDENT LCHEV IDENT RCHEV ZWSP RCHEV
```

The parser can then distinguish the binary operator ">>" from the above.

The parser generator Menhir

- The LIGO pipeline is more than a compiler, because it provides support for a LSP, and that is why the front-end has to keep all the source information, including comments. This adds significant complexity to the lexer, which has to hide it from the parser.
- The two LIGO parsers are generated by Menhir.
- Menhir accepts LR(1) grammars, with semantic actions in OCaml. In practice, many grammars are annotated with precedence declarations to solve LR-conflicts, potentially making the grammars not LR(1), and making it difficult to reason about them.
- Both **CameLIGO** and **JsLIGO** are given pure LR(1) grammars, thanks to the rewriting of productions, and the use of virtual tokens.
- Yann Régis-Gianas wrote the front-end of Menhir, and François Pottier the automaton construction and the back-end. The original interest was an application of GADTs to the static typing of the parse stack: <https://cambium.inria.fr/~fpottier/publis/fpottier-regis-gianas-typed-lr.pdf>

- Menhir can identify all error states in the LR-automaton, so we can write precise error messages for them. They are precise because the stack and syntactic right-context is available at the error state, instead of just only the next valid tokens, so the past (the intention of the programmer, inferred from the prefix of the stack in terms of grammatical productions) can be expressed in terms of constructs, and the future too, if we take some care when writing the grammar. <https://cambium.inria.fr/~fpottier/publis/bour-pottier-reachability.pdf>
- The fun part of the JsLIGO grammar is the handling of the (in)famous *Automatic Semicolon Insertion (ASI)*. JavaScript has a set of dynamic heuristics allowing in some contexts to omit a semicolon. Instead, I analysed the grammar and hardcoded a sub-grammar (~100 LOC) for statements that allow some semicolons to be missing (and omits more than the mainstream compilers!).
- One last very cool paper about using Menhir to parse C11 (as in CompCert): <https://cambium.inria.fr/~fpottier/publis/jourdan-fpottier-2016.pdf>

- Menhir has several interfaces, improperly called APIs. The *Incremental API* enables the client to take control when an error is encountered.
- The parse stack (of terminal and non-terminals recognised so far) and the automaton state (e.g. the LR-items) are available for inspection, as persistent values.
- This enables to decide to either handle the error or try and recover from it: a strategy has to be provided by the client.
- LIGO features error recovery for both **CameLIGO** and **JsLIGO**. Two continuations are possible for recovery: either skipping some tokens or inserting one. The decision is made based on a heuristic derived from a study of the grammar and common cases (design patterns, keywords and symbol uses, tabulations etc.).
- The LSP makes constant use of this feature to maintain syntax high-lighting, to display types on hover, and for go-to-definition on contracts being edited.

- The parser produces an Abstract Syntax Tree (AST) — confusingly named Concrete Syntax Tree (CST) in the code base for legacy reasons: it used to be what its name proclaims.
- The AST is then transformed into a more abstract kind of tree, the *unified AST*. Whereas the CSTs are specific to each LIGO variant, the unified AST is common to all, and can be considered as the entrance to the common pipeline.
- Because it is the target of a functional and imperative language, respectively **CamelLIGO** and **JsLIGO**, the definition of the Unified AST is cumbersome. Also, in order to enable some measure of decompilation (type expressions for the LSP), it cannot be too abstracted.

JsLIGO concrete syntax

```
1: type parameter = unit;
2: type storage = unit;
3: type t = [list<operation>, storage];
4:
5: @entry
6: function give5tez (param: parameter, storage: storage): t {
7:   let operations: list<operation> = [];
8:   if (Tezos.get_balance() >= 5tez) {
9:     const receiver =
10:       match(Tezos.get_contract_opt(Tezos.get_sender())) {
11:         when(Some(contract)): contract;
12:         when(None): failwith("Couldn't find account");
13:       };
14:     operations =
15:       [Tezos.Operation.transaction(param, 5tez, receiver)];
16:   }
17:   return [operations, storage];
18: }
```

Even though the syntax error messages displayed by the TypeScript Playground, for example, are relatively good, the JsLIGO parser enables to be correct about the past and complete about the future. For instance:

File "syn_err.jsligo", line 2, characters 9-11:

```
1 | interface I {  
2 |   type t += int;  
3 | }
```

Ill-formed type declaration in an interface.

At this point, one of the following is expected:

- * the assignment symbol '=' followed by a type;
- * type parameters (variables) between chevrons ('<', '>') if the type is generic;
- * a semicolon ';' if the type is abstract;
- * a closing brace '}' if no more entries.

JsLIGO versus TypeScript

- Despite the subset of TypeScript needed for writing JsLIGO contracts is comfortable enough (some computations can be expressed in several ways, for instance, and insertion of Michelson code is possible), a syntax error cannot tell whether the input is actually valid TypeScript or not.
- For example, in the previous example, the parser cannot actually tell that the input is not valid TypeScript, and the penultimate example is valid JsLIGO but not correct TypeScript: JsLIGO is a non-conservative extension of TypeScript.
- Likewise, a type error cannot tell whether the source is valid TypeScript. Worse, since the LIGO type system is *ad hoc*, the inferred types may differ from those of a TypeScript compiler (for example, due to different implicit type coercions).
- The LIGO tooling (mainly the LSP) is custom-made.

In short: Idioms and design patterns commonly used by TypeScript developers are not supported in JsLIGO; it is not possible for the LIGO compiler to tell whether it is because the input is not valid TypeScript. If the code compiles, but the semantics differs, the LIGO team would consider it a bug, but it could be silent (we never found an example, though).

The parser generator tree-sitter

- In order to leverage existing TypeScript tooling, for instance, a pretty-printer,
- to have the LIGO compiler inform the developers of the differences between TypeScript and JsLIGO,
- and to avoid writing a lexer and parser for TypeScript with ocamllex and Menhir, a new approach was proposed, namely use a third-party TypeScript parser.
- Since those are most often closed-source or blatantly incomplete, we decided to use a parser in C generated by tree-sitter from a TypeScript grammar.
- It is difficult to assess the correctness, let alone the conformance, of that parser.
- First, contrary to Menhir, which has a Coq back-end, we do not have a way to prove that the parser indeed recognises the language generated by the grammar, and is complete, that is, exactly the valid inputs are accepted. (Fun/Sad fact: termination is not proved with "menhir --coq"!)
- Second, there is no more formal grammar of TypeScript published by Microsoft since version 1.8 (current is 5.8.3) — in 2016.

The parser generator tree-sitter

- tree-sitter has been designed to generate parsers in C from LR(1) grammars.
- Those parsers perform *error recovery*: when encountering an unexpected token, the parser can either *skip* any number of tokens or non-terminal symbols from the input, or *insert* one virtual terminal (token) when one is deemed missing in the input.
- Instead of choosing one strategy rather than the other, the parser engine runs internally as a *Generalised LR* (GLR) algorithm, so, in general, the parse stack is instead a tree of valid prefix sentences (a.k.a. sentential forms). Skipping and inserting become two new branches (up to six).
- Each branch of the tree maintains two quantities: an *error cost* and a *node count*, respectively, an integer function of the number of skipped items and inserted tokens, and the number of valid subtrees parsed since the last error.
- One heuristic prunes the automaton's tree based on those two values, and another selects one final parse tree if many valid were found (smaller and earlier trees are preferred).